



CP Bootcamp 2026 — Day 14

Theme: Graph Foundation + BFS/DFS

Main Goal:

আজকে তুমি এমন একটি data structure শিখবে যেটা বাস্তব পৃথিবীর অনেক problem represent করে।

Examples:

`id="3x8f8n"`

People Connection

Road Network

Social Network

Computer Network

Internet

এগুলো সব Graph দিয়ে represent করা যায়।

আজকের focus:

Graph

↓

Vertex + Edge

↓

Graph Representation

↓

DFS

↓

BFS

↓

Graph Problem Thinking

Estimated Time: 6–7 hours



Day 14 Chapter Map

Day 14

- Chapter 1 – Graph Mental Model
- Chapter 2 – Graph Terminology
- Chapter 3 – Graph Representation
- Chapter 4 – Depth First Search (DFS)
- Chapter 5 – Breadth First Search (BFS)
- Chapter 6 – Connected Components
- Chapter 7 – Grid Graph Thinking
- Chapter 8 – Graph Problem Patterns
- Chapter 9 – Common Bugs & Edge Cases
- Chapter 10 – Day 14 Assignment & Practice

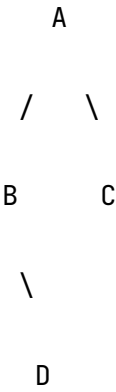
Chapter 1 — Graph Mental Model

Graph কী?

Graph হলো:

এমন একটি data structure যেখানে nodes (vertices) এবং connections (edges) থাকে।

Structure:



এখানে:

Nodes:

A B C D

Connections:

A-B

A-C

B-D

Real Life Example

Facebook

Person:

Node

Friendship:

Edge

Google Maps

City:

Node

Road:

Edge

Internet

Computer:

Node

Connection:

Edge

Chapter 2 — Graph Terminology

Vertex (Node)

Graph-এর element।

Example:

A

একটি vertex।

Edge

দুই vertex-এর connection।

Example:

A ---- B

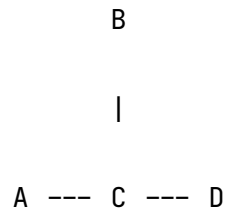
এখানে:

A-B = Edge

Degree

একটি node-এর কয়টি connection আছে।

Example:



C-এর degree:

3

কারণ:

A

B

D

এর সাথে connected।

Directed Graph

Connection-এর direction আছে।

Example:

Twitter follow:

$A \rightarrow B$

মানে:

A follows B.

কিন্তু B necessarily A-কে follow করে না।

Undirected Graph

দুই দিকেই connection।

Example:

Friendship:

A --- B

Weighted Graph

Edge-এর value থাকে।

Example:

Road:

Dhaka ---- Chittagong

250 km

Weight:

250

Unweighted Graph

শুধু connection আছে।

Example:

A --- B

Chapter 3 — Graph Representation

Computer graph বুঝে না।

তাই represent করতে হয়।

মূলত ৩টি method:

1. Adjacency Matrix
2. Adjacency List
3. Edge List

1. Edge List

সব edge store করি।

Example:

Graph:

A --- B

A --- C

B --- D

Store:

(A,B)

(A,C)

(B,D)

Simple

কিন্তু neighbor খুঁজতে slow

2. Adjacency Matrix

Matrix:

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	0
D	0	1	0	0

Meaning:

1 = Connected

0 = Not Connected

Advantages:

Connection check:

$O(1)$

Problem:

Memory:

$O(V^2)$

Large graph-এর জন্য expensive।

3. Adjacency List

সবচেয়ে বেশি CP-তে ব্যবহার হয়।

Example:

Graph:

A --- B

A --- C

B --- D

Representation:

A → B C

B → A D

C → A

D → B

C Implementation:

```
int graph[100][100];
```

বা:

```
vector<int> graph[100];
```

(C++ এ বেশি use হয়)

C-তে:

```
struct Node
{
    int vertex;

    struct Node* next;
};
```

কেন Adjacency List?

কারণ:

Most graphs:

Few connections

মানে:

Sparse graph।

Memory:

$O(V+E)$

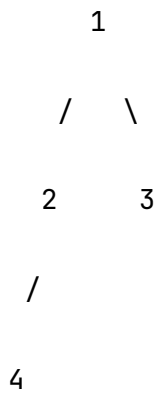
Chapter 4 — Depth First Search (DFS)

DFS:

যত গভীরে যাওয়া যায়, আগে যাও।

Example:

Graph:



DFS:

Possible:

1 2 4 3

DFS uses:

Recursion

or

Stack

DFS Mental Model

Visit Node

↓

Mark Visited

↓

Go Deeper

↓

Return

Visited Array

Graph-এ cycle থাকতে পারে।

Example:

A --- B

\ /

C

Visited না রাখলে:

Infinite loop।

Use:

```
int visited[100];
```

DFS Code Idea

```
void dfs(int node)
{
    visited[node]=1;

    printf("%d ",node);

    for(each neighbour)
    {
        if(!visited[neighbour])
        {
            dfs(neighbour);
        }
    }
}
```

DFS Example

Start:

1

Flow:

1

↓

2

↓

4

↓

back

↓

3

DFS Applications

Use:

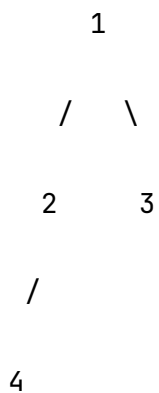
- Connected components
- Cycle detection
- Maze solving
- Path finding

Chapter 5 — Breadth First Search (BFS)

BFS:

Level by level visit করে।

Example:



BFS:

1 2 3 4

BFS uses:

Queue

কারণ:

First discovered node আগে process হবে।

BFS Mental Model

Put Start Node

↓

Take Front

↓

Visit Neighbours

↓

Add New Nodes

BFS Algorithm

```
queue.push(start)
```

```
visited[start]=1
```

```
while queue not empty:
```

```
    node = queue.front()
```

```
    remove node
```

```
    visit neighbours
```

```
    add unvisited neighbours
```

BFS Code Idea

```
void bfs(int start)
{
    queue[start];

    visited[start]=1;

    while(queue not empty)
    {
        node=remove();

        for(each neighbour)
        {
            if(!visited[neighbour])
            {
                visited[neighbour]=1;

                add(neighbour);
            }
        }
    }
}
```

DFS vs BFS

DFS	BFS
Depth first	Level first
Stack/Recursion	Queue

DFS	BFS
Maze	Shortest path
Deep exploration	Near exploration

Chapter 6 — Connected Components

Problem:

Graph:

A --- B

C --- D

এখানে:

দুইটা আলাদা group আছে।

Connected Component:

একটি connected group।

Count Components:

Idea:

count=0

সব node:

unvisited?

তাইলে:

count++

DFS/BFS

Algorithm:

for every node:

if not visited:

component++

DFS(node)

Example:

Graph:

1---2

3---4

5

Components:

3

Chapter 7 — Grid Graph Thinking

অনেক CP problem:

Grid:

1 1 0

0 1 0

1 0 1

আসলে graph।

Each cell:

Node

Neighbour:

Edge

Example:

Island Problem:

1 1 0

1 0 0

0 0 1

প্রতিটি connected 1:

একটি island।

Need:

DFS/BFS।

Directions:

4 direction:

up

down

left

right

Code idea:

```
int dx[4]={-1,1,0,0};
```

```
int dy[4]={0,0,-1,1};
```

Chapter 8 — Graph Problem Patterns

Pattern 1 — Traverse All Nodes

Problem:

"Visit every node"

Think:

DFS/BFS

Pattern 2 — Shortest Path in Unweighted Graph

Example:

Minimum number of moves।

Think:

BFS

কারণ BFS level অনুযায়ী যায়।

Pattern 3 — Count Groups

Example:

Number of islands

Think:

DFS/BFS + Visited

Pattern 4 — Detect Cycle

Think:

DFS

+

Visited State

Pattern 5 — Grid Problems

Think:

Grid = Graph

Cell = Node

Chapter 9 — Common Bugs & Edge Cases

Bug 1 — Visited না রাখা

Problem:

Cycle হলে:

Infinite recursion।

Bug 2 — Directed vs Undirected Confusion

Undirected:

A-B হলে:

$A \rightarrow B$

$B \rightarrow A$

দুইটা add করতে হয়।

Bug 3 — Wrong Neighbor Loop

Adjacency list ঠিকমতো traverse না করা।

Bug 4 — Grid Boundary

Wrong:

```
grid[x][y]
```

আগে check না করে।

Need:

$$0 \leq x < \text{rows}$$
$$0 \leq y < \text{cols}$$

Bug 5 — BFS-এ Queue ভুল

BFS:

Stack না।

Need:

Queue

Chapter 10 — Day 14 Assignment

Part A — Graph Representation

Implement:

1. Store Edge List
2. Build Adjacency Matrix
3. Build Adjacency List

Part B — DFS

Implement:

1. DFS traversal
2. Count connected components
3. Detect cycle

Part C — BFS

Implement:

1. BFS traversal
2. Shortest distance in unweighted graph
3. Level order traversal

Part D — Grid Problems

Solve:

1. Count Islands
2. Flood Fill
3. Maze Path

Part E — Dry Run

Graph:

1 --- 2

|

3 --- 4

Start:

1

Write:

DFS order:

?

BFS order:

?



Glossary Update

Add:

Graph

Vertex

Node

Edge

Degree

Directed Graph

Undirected Graph

Weighted Graph

Adjacency Matrix

Adjacency List

DFS

BFS

Visited Array

Connected Component

Cycle

Grid Graph



Pattern Library Update

Add:

DFS Pattern

Recognition:

Need to explore all connected nodes.

Mental Model:

Visit

↓

Mark

↓

Go Deeper

↓

Return

BFS Pattern

Recognition:

Need shortest path in unweighted graph.

Mental Model:

Queue

↓

Level by Level

↓

Expand Neighbours

Grid as Graph Pattern

Recognition:

- Island problems
- Maze
- Flood fill

Mental Model:

Cell = Node

Direction = Edge



Day 14 Completion Checklist

- [] Graph concept বুঝি
- [] Vertex এবং Edge বুঝি
- [] Directed/Undirected বুঝি
- [] Graph representation বুঝি
- [] Adjacency List বুঝি
- [] DFS implement করতে পারি
- [] BFS implement করতে পারি
- [] Visited array বুঝি
- [] Connected component বুঝি
- [] Grid কে graph হিসেবে ভাবতে পারি
- [] Shortest path-এর জন্য BFS চিনতে পারি



Day 14 Final Mental Model

Problem



Relationship আছে?



Graph



Need explore?



DFS/BFS



Need shortest?



BFS



Need groups?



Connected Components

Day 14 শেষে:

Day 1-5
Pattern Foundation

+

Day 6
Functions

+

Day 7
Searching

+

Day 8
Sorting

+

Day 9
Optimization

+

Day 10
Recursion

+

Day 11
Stack Queue

+

Day 12
Linked List + Hashing

+

Day 13

Trees

+

Day 14

Graphs

পরের:

Day 15 — Dynamic Programming Foundation + Problem Solving Strategy

এটা Bootcamp-এর সবচেয়ে important শেষ দিকের chapterগুলোর একটি হবে।